

**ЛЕКЦИЯ 6.2**  
**СЪСТАВНИ/СТРУКТУРИРАНИ**  
**ТИПОВЕ ДАННИ: НИЗОВЕ**

# АТД НИЗ: АМ

**Низът** (*string\**) представлява подредена съвкупност от краен брой символи (знаци). Това е статичен АТД\*\*, тъй като броят на неговите елементи (символите) не може динамично да се променя по време на съществуване на структурата. До компонентите на низа е осигурен директен достъп

- **Основни понятия и характеристики:**

- Броят на елементите на даден низ се нарича **дължина на низа**
- **Базовият тип** на низа е символният тип
- По това, че компонентите на низа са еднотипни и по директния достъп до тях, този тип много прилича на едномерните масиви

- **Приложение:**

- Моделиране на данни, които представляват текст на някой естествен език (основно приложение)
- Определяне на множеството на входни и изходни данни на описвания алгоритъм (въвеждане и извеждане на информация)

\*Да не се бърка с класа *String* тук се има предвид просто като понятие за тип данни в ЕП

\*\* при различни ЕП имплементацията може да е различна или да съществува повече от една имплементация на този тип данни

# АТД НИЗ: ДОСТЪП, КОНСТРУИРАНЕ И ДРУГИ ОПЕРАЦИИ

- **Достъп:**

Приликата между низовете(**Character array**) и масивите определя и същия механизъм на избор – елементите на низа са достъпни пряко **чрез техните индекси** (относителното място в низа)

- **Конструиране:**

Низовете се конструират просто **чрез изброяване** на символите (елементите на низа). Символите на един низ се ограждат в някакъв вид “*скоби*”, обикновено това са “...” или ‘...’

- **Кодиране:**

Символите в низа се кодират по някой стандарт, например ASCII, ISO/IEC 8859, Unicode

# НИЗОВЕТЕ В ЕЗИЦИТЕ ЗА ПРОГРАМИРАНЕ

- **Типични характеристики** в ЕП от високо ниво:
  - Низовете са предварително дефинирани като стандартен тип-string
  - Низовете се реализират просто като едномерни масив от символи
- **Дефиниране** (Java) – декларира се с или без максималният брой знаци:

```
char[] JCharArray;
```

```
char[] JCharArray = new char[4];
```

- **Конструиране** – в повечето случаи съществуват два подхода:
  - като се изброяват символите:

```
JCharArray[0] = 'r';
```

```
JCharArray[1] = 's';
```

- като се използват вградените средства(1) или обхождане с някой от операторите за цикъл.

```
(1) Scanner sc=new Scanner(System.in);  
    System.out.println("Please enter elements...");  
    char[] a=sc.next().toCharArray();
```

- **Достъп :**

- като се използва означение чрез индекси за всеки отделен елемент:

```
System.out.println(JCharArray[0]);
```

# АТД НИЗ (CHARACTER ARRAY) В C++: ФМ И ЗМ

- **Физически модел**

В паметта на компютъра всяка константа (стойност) от тип низ се записва в толкова байта, колкото са символите на низа +1. **Не може да се променя размера-след.** Символните низове са статична структура.

- **Знаков модел**

- C++ няма собствен тип “низ” – използват се C базирани. Използва се масив от тип `char` с **нула за край** – `'\0'` (*Null-terminated*)
- За да се опрости обработката на низове C++ предоставя набор от функции за обработка на низове, намиращи с във файла `<cstring>` (`string.h`)

- **Синтаксис за деклариране на променлива-низ:**

`char <име> [ <константен израз> незад ];`

, където:

- `<име>` е името на променливата-низ
- `<константен израз>` е от целочислен тип и има стойност  $N \geq 0$  и определя максималният брой знаци в низа, който поради допълнителния символ за край трябва да бъде с 1 по-голям от действителния брой знаци в низа
- `<константен израз>` може да се пропусне само ако е извършено инициализиране на низа или ако той е деклариран като параметър на функция

- **Примери:**

```
char s[7]; //низ с 6 елемента + '\0'
```

# АТД НИЗ В C++: ЗМ – МНОЖЕСТВО ОТ СТОЙНОСТИ, ОПЕРАЦИЯ СЕЛЕКТОР

- **Множество от стойности:**

Съвкупност от всички възможни последователности от символи на езика, дължината на които не надхвърля декларираната максимална дължина, включително и празната последователност

Последователности от символи, оградени от двойни кавички се приемат като **низови константи**: "qwerty", "%50 xyz", "1"

- **Операцията селектор** за стойности от тип низ прилича на операцията за достъп до елементите на масив, осъществява се чрез индекса:

```
char strng[10];  
strng[0]='a';  
cout << strng[0];
```

# АТД НИЗ В C++: ЗМ – КОНСТРУКТОР И ИНИЦИАЛИЗИРАНЕ

- **Конструктор и инициализиране** – по начин подобен на масивите:

## 1. Чрез обхождане (*traversing*)

```
char s[10];  
s[0]='a';   s[1]='b';   s[2]='\0';
```

## 2. При инициализиране – низовете могат да бъдат инициализирани по няколко начина:

```
(1) char S [5] = { 'a', 'b', 'c', 'd', '\0' };  
(2) char S[5] = "Boom";  
(3) char s[10]; cin>>s;
```

При (2) начин компилаторът сам добавя символа за край

' \0 ' (препоръчва се)

# АТД НИЗ В C++: ОПЕРАЦИИ И РЕЛАЦИИ - НАСЛЕДЕНИ ОТ C

- **Въвеждане и извеждане:** Низовете могат да се въвеждат и извеждат с помощта на `cin` и `cout`, както и с помощта на `printf` и `scanf`
- **Всички останали операции и релации** с низове са реализирани като стандартни функции, чиито прототипи се намират в `<cstring>` (`string.h`). Най-важните от тях са:

| Ф у н к ц и я        | П р е д н а з н а ч е н и е                                                     |
|----------------------|---------------------------------------------------------------------------------|
| <code>strcat</code>  | С лепва един низ към друг                                                       |
| <code>strchr</code>  | Н а м и р а п ъ р в о т о с р е щ а н е н а д а д е н с и м в о л в н и з       |
| <code>strcmp</code>  | С равнява два низа                                                              |
| <code>strcpy</code>  | К о п и р а е д и н н и з в д р у г                                             |
| <code>strlen</code>  | Н а м и р а д ъ л ж и н а т а н а н и з                                         |
| <code>strncat</code> | Д о б а в я с и м в о л и о т е д и н н и з в д р у г                           |
| <code>strncmp</code> | С равнява символи от два низа                                                   |
| <code>strncpy</code> | К о п и р а с и м в о л и o t e d и н н и з в d r u g                           |
| <code>strrchr</code> | Н а м и р а п о с л е д н о т о с р е щ а н е н а д а д е н с и м в o л в n i z |
| <code>strstr</code>  | Н а м и р а п ъ р в о т о с р е щ а н е н а д а д е н н и з в d r u g           |

- **Пример:**

```
strcpy(strng, "abs");
```

Запомнете, че присвояване на низове не е реализирано  
Често срещана грешка е препълване на низа

# АТД НИЗ В C++: СТАНДАРТНИ ФУНКЦИИ ЗА ПРЕОБРАЗУВАНЕ - НАСЛЕДЕНИ ОТ C

- Стандартните функции, чрез които можем да извършваме преобразуване между тип низ и други стандартни типове са реализирани в `<cstdlib>` (`stdlib.h`). Най-важните от тях са:

| Ф у н к ц и я       | П р е д н а з н а ч е н и е                                     |
|---------------------|-----------------------------------------------------------------|
| <code>atof</code>   | П р е о б р а з у в а <code>string</code> в <code>float</code>  |
| <code>atoi</code>   | П р е о б р а з у в а <code>string</code> в <code>int</code>    |
| <code>atol</code>   | П р е о б р а з у в а <code>string</code> в <code>long</code>   |
| <code>_itoa</code>  | П р е о б р а з у в а <code>int</code> в <code>string</code>    |
| <code>_ltoa</code>  | П р е о б р а з у в а <code>long</code> в <code>string</code>   |
| <code>_ecvt</code>  | П р е о б р а з у в а <code>double</code> в <code>string</code> |
| <code>strtod</code> | П р е о б р а з у в а <code>string</code> в <code>double</code> |

# АТД НИЗ В C++: НИЗОВЕТЕ КАТО CHAR



- В C++ низовете са масиви от `char` елементи, но от своя страна масивите са тясно свързани с указателите, ето защо следващите дефиниции са еквивалентни:

```
char  
s[7]="abc";
```

И

```
char *s="abc";
```

```
int getline(char s[],int lim)  
{  
    ...  
    s[i]=c;  
    ...  
}
```

И

```
int getline(char *s, int lim)  
{  
    ...  
    *(s+i)=c;  
    ...  
}
```

# АТД НИЗ В C++: ПРИМЕР ЗА РЕАЛИЗАЦИЯ НА ФУНКЦИЯТА STRCPY

**Примерна** реализации на strcpy, която илюстрира не само как масиви от символи с нула за край се използват като низове, но и как тези две представяния могат да се заменят взаимно. Функцията strcpy, която копира низа f (от) в низа t (към):

```
void strcpy(char t[], char f[])
{
    int i=0; char c;
    c=f[i];
    t[i]=c;
    while(c!='\0') {
        c=f[i];
        t[i]=c;
        i=i+1;
    }
}
```

# АТД НИЗ В C++: ОСНОВНИ АЛГОРИТМИ С НИЗОВЕ

- Замяна на подниз от даден низ с друг
- Изтриване на подниз от даден низ
- Търсене на подниз в даден низ
- Подреждане на низове в азбучен ред
- Определяне дали даден низ е палиндром.

**Палиндром** – “алена фанела”, „аз обичам мач и боза”  
(виж функцията `strrev`)

# АТД НИЗ В C++: СТАНДАРТНИ БИБЛИОТЕКИ НА C++

- STL съдържа класа `string`, който предоставя много по-сигурна реализация на абстракцията низ на високо ниво, които са включени в заглавния файл `<string>`
- Тук низът вече е динамична структура, затова са налични функции

като(<http://www.cplusplus.com/reference/string/string/>):

|                                      |                                                        |
|--------------------------------------|--------------------------------------------------------|
| <b>operator+=</b>                    | Append to string (public member function )             |
| <b>append</b>                        | Append to string (public member function )             |
| <b>push_back</b>                     | Append character to string (public member function )   |
| <b>assign</b>                        | Assign content to string (public member function )     |
| <b>insert</b>                        | Insert into string (public member function )           |
| <b>erase</b>                         | Erase characters from string (public member function ) |
| <b>replace</b>                       | Replace portion of string (public member function )    |
| <b>swap</b>                          | Swap string values (public member function )           |
| <b>pop_back</b> <small>C++11</small> | Delete last character (public member function )        |

# STRING CLASS

- Strings са **обекти** представлящи последователности от символи.
- Декларация - пример: `string grtng = "Hello Petar";`
- Основни операции: **Задание - изследвайте литературните източници**
- **пример - конкатенация** (оператор „+“ или функция `append()`)

```
string firstName = "Ivan";  
string lastName = "Petrov";  
string fullName = firstName.append(lastName);  
cout << fullName;
```

- Тъй като се изискват познания и по ООП не се разглеждат подробно тук.